COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Baltazar, Kenji Formilleza, Huone Anthony	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B

Part 1

CODE

```cpp
#include <iostream>
#include <thread>

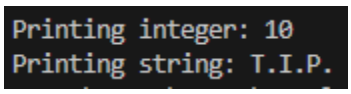
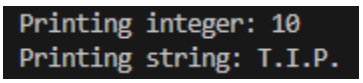
void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

Output

BALTAZAR	FORMILLEZA

	 
Analysis	

BALTAZAR	FORMILLEZA
For this section, the single thread example executes just as predicted. I ran the code multiple times and the output does not change due to the fact that there is only one thread in use. In this case, the integer and string are printed in sequence and are dependent on their line of code. As if it were otherwise, the output would also be reversed that the concurrent output.	This program starts a new thread that runs the print function with 10 and "T.I.P." as its arguments. The join call makes main wait for that thread to finish before the program ends. The output is two lines, one for the integer and one for the string.

Part 2

CODE

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

Output	

BALTAZAR	FORMILLEZA
0 : T.I.P. 2 : Computer 3 : Engineers 1 : Competent	0 : T.I.P.3 : Engineers2 : Computer 1 : Competent

Analysis	

	BALTAZAR	FORMILLEZA
	In this section, the output is completely different from the predicted sequence of per line of code. Proof of that is the output given, starting from 0 but skips to 2, proceeding to 3, then 1 has been executed. I think this is due to the fact that each number of threads from the for loop of join finished differently. When in comparison to the output my partner provided, mine seemed more in order and neat, thus proving further that when we are multithreading, the output is not guaranteed to be the same and some output may seem more distorted than others because some threads may finish much more earlier than the other threads that precedes it or next to it.	In this code the output is different every time it is executed because it depends on which threads finishes first, each one saves its own message in its own slot in a shared list since each different thread writes in a different slot, they don't clash with each other. The main thread waits for all of them to join, then prints the list in order, so the output is the same every time.

2. SUPPLEMENTARY

Part A

Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.
-Multithreading means one program runs several threads at the same time. Threads share the same memory. In C++ we use std::thread. Advantages are faster programs on multicore CPUs and a program that stays responsive and disadvantages are race conditions, deadlocks, and bugs that are hard to find.

2. Rationalize the use of multithreading by providing at least 3 use-cases.
-GUI apps, so the window does not freeze while loading a file.
Servers, so many clients can be handled at once.
Sorting or processing big data by splitting the work between threads.

3. Differentiate between parallelism and concurrency.
-Concurrency means tasks take turns and all make progress, even on one core. Parallelism means tasks really run at the exact same time on different cores.

Part B

Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:
  - o Display
  - o T1.join()
  - o Display
  - o T2.join()
  - o Display
  - o T3.join()

Provide an analysis based on the outputs of the multi-threading exercise.

CODE

```cpp
#include <iostream>
#include <thread>
#include <chrono>

int globalValue = 0;

void add(int value){
    std::this_thread::sleep_for(std::chrono::milliseconds(value * 20));
    globalValue += value;
}

int main(){
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    std::cout << "Value = " << globalValue << std::endl;
    t1.join();
    std::cout << "Value = " << globalValue << std::endl;
    t2.join();
    std::cout << "Value = " << globalValue << std::endl;
    t3.join();
    std::cout << "Value = " << globalValue << std::endl;

    return 0;
}
```

OUTPUT

```
Value = 0
Value = 10
Value = 30
Value = 60
```

ANALYSIS

The first print shows 0 because the threads are still running when main prints. Join makes main wait for a thread to finish, so the value becomes 10 after T1 join, 30 after T2 join, and 60 after T3 join. Only the last print is always 60, while the others depend on timing. The three threads ran at the same time, so the program took about 600 ms and not 1200 ms. If many threads change the same variable at once, a mutex is needed to avoid race conditions.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

CODE

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <cstdlib>

std::vector<int> arr;

void mergeParts(int left, int mid, int right){
    std::vector<int> temp;
    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right){
        if (arr[i] <= arr[j]){
            temp.push_back(arr[i]);
            i++;
        }
        else{
            temp.push_back(arr[j]);
            j++;
        }
    }
    while(i <= mid){
        temp.push_back(arr[i]);
        i++;
    }
    while(j <= right){
        temp.push_back(arr[j]);
        j++;
```

```
31      }
32      for(int k = 0; k < (int)temp.size(); k++){
33          arr[left + k] = temp[k];
34      }
35  }
36
37  void mergeSort(int left, int right){
38      if (left >= right)
39      {
40          return;
41      }
42      int mid = (left + right) / 2;
43      mergeSort(left, mid);
44      mergeSort(mid + 1, right);
45      mergeParts(left, mid, right);
46  }
47
48  void threadedMergeSort(int left, int right){
49      int mid = (left + right) / 2;
50      std::thread t1(mergeSort, left, mid);
51      std::thread t2(mergeSort, mid + 1, right);
52
53      t1.join();
54      t2.join();
55
56      mergeParts(left, mid, right);
57  }
58
59
60  int main(){
```

```
    int n = 1000000;
    std::vector<int> original(n);
    for (int i = 0; i < n; i++){
        original[i] = std::rand() % 100000;
    }
    arr = original;
    auto start1 = std::chrono::high_resolution_clock::now();
    mergeSort(0, n - 1);
    auto end1 = std::chrono::high_resolution_clock::now();

    arr = original;
    auto start2 = std::chrono::high_resolution_clock::now();
    threadedMergeSort(0, n - 1);
    auto end2 = std::chrono::high_resolution_clock::now();

    double time1 = std::chrono::duration<double, std::milli>(end1 - start1).count();
    double time2 = std::chrono::duration<double, std::milli>(end2 - start2).count();

    std::cout << "n = " << n << std::endl;
    std::cout << "Single thread: " << time1 << " ms" << std::endl;
    std::cout << "Two threads: " << time2 << " ms" << std::endl;


    return 0;
}
```

OUTPUT

```
n = 1000000
Single thread: 891.552 ms
Two threads: 411.911 ms
```

ANALYSIS

The two thread version should be faster on a big array because both halves are sorted at the same time. It is not twice as fast because the last merge is still done by one thread, and making threads also takes some time. For a small array the threads would not help at all. Change this to match your own screenshot if your two thread time came out slower.

3. CONCLUSION

This activity helped us see how multithreading really works instead of just reading about it. I thought threads would simply make everything faster, but it turns out there is more to it. I learned how join makes the main thread wait, and in Part B the value went from 0 to 10 to 30 to 60 as each thread finished. In Part C, splitting merge sort into two threads only helped when the array was big enough, since small ones were not worth the time of making threads. Overall we think we did okay since the programs worked, but we still need to work on using more threads and on locks and deadlocks.

COMPUTER ENGINEERING

QUEZON CITY

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **We use these tools to help us fix our errors in our code and paraphrase our sentences.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used only to help us fix errors in our code as well as paraphrase some of our sentences. It was not used to an extent where it replaces our original thinking, our analysis and the completion of this work is our own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.